Interaktivt webbprojekt - Translators AB

Information:
I html-filen har jag adderat extra, fiktiva element för att ha mera att analysera än den ursprungliga koden.
Bland annat den i CSS stylade Cookie bannern, som i uppgiften inte visas som pop-upp utan bara finns visuellt.

Säkerhet

Sårbarhetsrisk

1. XSS

2. Osäker lagring av lösenord

3. Bristande validering av användarinmatning

Om webbplatsen skulle ha användarkonton bör lösenord aldrig lagras i klartext. I stället bör de hash-as och saltas, exempelvis med algoritmer som bcrypt.

Inmatningsfältet #inputMeddelande - risk för DOM-baserad XSS.
DOM-baserad XSS kan uppstå. Ingen server är inblandad, datan skulle kunna komma via ingripandens inmatning i input-fältet, JS i användarens webbläsare skulle kunna stoppa in en skadlig kod in i sidan via .innerHTML och webbläsaren tolka det som kod och manipulera DOM.

Problem fanns i koden:

```
document.getElementById("output").innerHTML = meddelande;
```

En DOM-baserad attack skickas aldrig till servern och är svåra att upptäcka av brandväggar för webbaserade program (WAF) och syns inte i serverloggar.[1]

Åtgärder

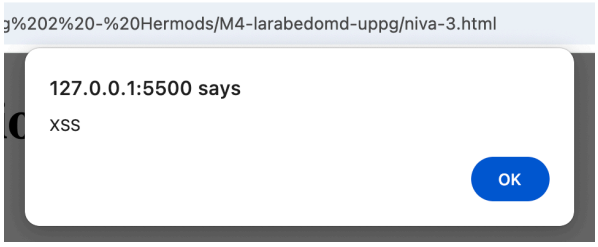
1. Först testade jag vanlig användar-/text-input i inmatningsfältet för att testa funktionen.

2. Därefter matade jag in:

a) <script>alert('Du har blivit hackad!')</script>
Chrome visar inte någon alert popp up men input via .innerHTML. Input tolkas här som HTML-kod och webbläsaren bygger DOM-element av det. Detta syns i Dev Tools:

```
<button onclick="visaMeddelande()">Visa</button>
...  ▼<div id="output"> == $0
      <script>alert('Du har blivit hackad!')</script>
```

Skriptet körs inte automatiskt eftersom det finns säkerhetsskydd i moderna webbläsare. Däremot kan HTML-attribut, som innehåller JS, med händelser som operor, onclick och oljad fortfarande triggas.

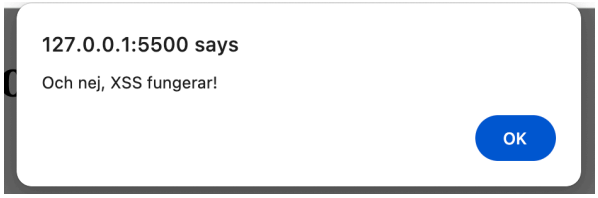


Därför testade jag också: <IMG SRC=/

[1] Vad är en XSS-attack? Definition och förklaring. https://www.kaspersky.se/resource-center/definitions/what-is-a-cross-site-scripting-attack (hämtad 2026-03-14)

onerror="alert(String.fromCharCode(88,83,83))"></img>[2]

och <img src=x onerror="alert('Och nej, XSS fungerar!')">



<img onerror>- testet fungerar som XSS. När element skapas ser webbläsaren att det är ett script-element och innehållet skickas till JS-motorn och skriptet kan aktiveras automatiskt

Säkerhetsåtgärd 1 i JS.

I stället för innerHTML lägger jag in .textContent i JS, för då tolkas inte input som HTML utan som ren text.

document.*getElementById*("output").*textContent* = meddelande;

Det skapas inget DOM-element som kan trigga onerror.

När jag bytte från innerHTML till textContent i JS visades användarens inmatning fortfarande på sidan som vanlig text, men utan att tolkas som HTML. Det minskade risken för DOM-baserad XSS. När jag därefter lade till en Content Security Policy slutade knappen först att fungera, eftersom min kod använde inline-JavaScript i både onclick och ett <script>-block. CSP blockerade alltså denna typ av skriptkörning. För att få både funktion och säkerhet behöver JavaScript därför flyttas till en extern fil

[2] OWASP. XSS Filter Evasion Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/XSS_Filter_Evasion_Cheat_Sheet.html (hämtad 2026-03-14)

och händelsen kopplas med addEventListener. Detta ger ett bättre skydd i flera lager.

Säkerhetsåtgärd 2

Ett extra skyddslager är Content Security Policy (CSP) I <head>:

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';
img-src ‘self'; style-src 'self';">
```

CSP skyddar mot säkerhetshot som Cross-Site Scripting (där angripare försöker initierar skadlig kod), data-stöld (genom att begränsa vilka externa tjänster kan nås), malware-distribution (genom att hindra från att farlig kod laddas in).

CSP-regeln kontrollerar vilka resurser såsom skript, stilar m.m. webbplatsen får ladda genom att begränsa vilka resurser webbsidan får ladda. Resurser som laddas får bara ha samma ursprung/domän, medan resurser från externa servar är inte tillåtna som standard.
JS får bara köras från samma webbplats, extern JS blockeras. Bilder får bara laddas från samma webbplats, medan bilder från andra domäner blockeras. Stilar (CSS) får endast ha samma ursprung som webbsidan, och eftersom styles.css ligger i samma domän som resten av webbplatsen kommer det att godkännas av CSP:n. Font från extern källa behöver laddas ned eftersom det annars kommer att blockeras av CSP. Eventuella inline CSS-attribut blockeras också, oavsett om dessa placeras i <body> eller i <head>, och även om det finns förfaringssätt för att behålla CSS i html-dokumentet så rekommenderar jag att all CSS läggs i en egen fil.

Ytterligare skydd via CSP kan vara:

- Object-src ‘none’ - blockerar alla <object>, <embed> och <applet>. Inga pluginbaserade objekt får laddas alls. Tidigare var det Flash, Java applets och pdf-plugins.

- Base-uri ‘self’ - förhindrar att relativa länkar blir absoluta, och användaren omdirigeras till farliga platser utanför sidan. <base> får bara peka på sin egen domän.
- Form-action ‘self’ - ser till att formulär bara skickas till egen server.

En alltför strikt policy med script-src'self' blockerar externa skript helt, inklusive analysprogram såsom Google Analytics. För att ändå kunna ladda Analytics efter användarens samtycke behöver man uttryckligen tillåta scriptkälla i script-src. På så sätt blockeras fortfarande okända externa skript, medan den specifika och betrodda källan för analysverktyget tillåts.

I CSP-koden föreslås följande undantag:

- i Script-src - tillåter Analytics-skriptet att laddas från extern tjänst.
- i Connect-src - tillåter att sidan skickar analysdata till analystjänsten.
- i img-src - analysprogram kan ibland använda bild/piexalanrop.

Säkerhetsåtgärd 3

Som ytterligare skydd kan inmatningen valideras eller filtreras innan den visas, exempelvis genom att ersätta tecken som “<“ och “>” i JS-koden så att de inte kan tolkas som kod. Användarens input visas redan som text, så sanitize-funktionen är mer ett extra skyddslager.

Säkerheten bygger alltså på flera lager: säker DOM-hantering, begränsning av resurser via CSP och försiktig hantering av användarinmatning.

Tillgänglighetsanpassning

Vi byggande av webbsidan behöver de fyra Webb Content Accessibility Guidelines (WCAG) principerna finnas. All information och användargränssnitt på sidan behöver kunna uppfattas (“perceivable”), användare måste kunna hantera alla användargränssnitt och navigering (“operable”), och kunna förstå alla information och alla användargränssnitt på sidan (“understandable”), till exempel om det inträffar ett fel när användaren klickar på “Submit” button och användaren får ett felmeddelande, behöver hen kunna förstå vad felet betyder och vad man ska göra för att komma vidare med det.[3]

Navigering via <tab>
Man knappar oftast mellan länkar, knappar, formulärfält och andra interaktiva element. På den här sidan går det att tabba mellan URL-fältet, inputfältet och knappen. Sidan bör använda semantiskt HTML, såsom <button>, <section> med flera, och sidan har flera <html> element som förslag, vilka är att föredra framför icke-semanstisk HTML element (såsom t.ex.).

Cookie-bannern har gjorts mer tillgänglig genom att använda en dold rubrik. Rubriken är visuellt dold med en så kallad visually hidden-teknik men finns kvar i dokumentstrukturen för skärmläsare. På så sätt kan användare som navigerar via rubriker identifiera sektionen.
Sektionen kopplas till rubriken genom attributet aria-labelledby. Ett alternativ hade varit att använda aria-label, men metoden med rubrik ger en bättre semantisk struktur eftersom rubriker kan användas för navigering i skärmläsare. Tekniken att visuellt dölja text men behålla den för assistentiv teknik är i överensstämmelse med W3C. [4]
För <div id=“output"> använder jag attributet `aria-live="polite"` vilket gör att skärmläsare automatiskt läser upp förändringar i innehållet när JS uppdaterar elementet. Detta är viktigt vid dynamiskt innehåll eftersom användare annars inte får någon information om att något har ändrats. I det här exemplet placeras output direkt på sidan, mest för att visa det.

Fokus

[3] Belfrage Paul, Webbutveckling 1, 2025, s. 196

[4] Rekommenderad WCAG 2.1 teknik: https://www.w3.org/WAI/WCAG21/Techniques/css/C7

Den fjärde WCAG principen, robusthet, ställer krav på att sidans innehåll ska vara tillgänglig med hjälpmedel och andra användaragenter och att arbete med tillgänglighet behöver vara dynamisk och webbsidans teknik hänga med när dessa tekniker utvecklas.

I dagsläget, år 2026 använder användare sig av exempelvis skärmläsare NVDA (vid Windows).[5] Personer som använder skärmläsare kan ofta hoppa mellan landmarks, regioner, rubriker. När en sektion är märkt med `aria-label` så blir navigationen tydligare.

(Knappen med id="button-m" markerade jag först med aria-label för skärmläsare, vartefter jag tog bort markeringen eftersom jag bedömde att knappen redan har ett namn genom sin synliga text "Skicka", vilket är tillräckligt uttrycksfullt för en skärmläsare.)

Vanligtvis får icke-semantiska html-element såsom ingen fokus. Tabindex="0" bör användas när ett ickelement behöver förtydligas ytterligare.

Hantering av fokus för Cookie-bannern

För att säkerställa att användare som navigerar med tangentbord eller skärmläsare inte missar cookie-bannern märks sektionen märkts med `tabindex="-1".` Normalt kan en `<section>` inte ta emot fokus, men med detta attribut kan fokus flyttas programmatiskt med JS `(element.focus())` när sidan laddas.

När fokus flyttas till bannern via JS börjar skärmläsaren läsa upp innehållet i sektionen. I detta fall läses först rubriken upp, följt av texten och knapparna, exempelvis:

- "Cookies."
- "Vi behöver ditt samtycke för att använda analys-cookies".
- "Acceptera cookies".
- "Neka cookies."

Genom att kombinera detta med attributet `aria-labelledby` läser skärmläsaren automatiskt upp bannerns rubrik när fokus flyttas till sektionen. På så sätt uppmärksammas användaren direkt på att ett val om cookies behöver göras.

Eftersom värdet `-1` används hamnar inte bannern i den vanliga tabb-ordningen. Efter att

[5] NVDA skärmläsare för Windows, https://www.nvaccess.org/

användaren har gjort sitt val kan hen fortsätta navigera vidare på sidan utan att oavsiktligt tabba tillbaka till bannern. Cookie bannern försvinner efter valet, och minnet om valet kommer finnas i LocalStorage inför nästa besök.

Bilden på sidan har fått en alt-beskrivning.

Färgkontrast:

- - - text: #111111

  - - card-bg: #f5f5f5

  Contrast-ratio: xxxx

  (Testat på webaim.com)[6]

- - - text: #11111

  - - button-bg: rgb(250, 242, 211) /

  vilket uppfyller WCAG princip om att informationen ska vara upptäckbar.

  Radavstånd mellan linjer bör vara minst 1.5 (satt i <body> i CSS).

Hållbarhet

Energikrävande animation har tagits bort. Den anses dessutom inte fylla någon annan en eventuell en estetisk funktion, men tar för mycket av uppmärksamhet från innehållet i sidan och är inte ändamålsenligt. Vill man ha animationen kvar bör bakgrunder följa organisationens grafiska profil. Om bakgrundskortet (.card) tas bort eller dess bakgrund ändras behöver nya färgkontrast tester göras mellan text och bakgrund för att uppfylla krav på tillgänglighet.

Optimering av bilder

[6] Contrast Checker, https://webaim.org/resources/contrastchecker/

Bilder står ofta för den största datamängden på en webbsida. Jag laddade ned en bild med skrivmaskin på ett skrivbord i mediumformat (800 x 450) på 81KB och omvandlade denna via sqoosh.app till WebP för att minska filstorleken. Med Quality minskad till 75% blev och nytt filformat blev besparingen på 59% till 33KB. Bakgrundsbilden optimerades via sqoosh.app och kunde minskas utan för stor kvalitetsbortfall från 1.2MB i ursprunglig .jpg-format till 200KB som WebP. I .jpg kunde väger bilder 197kB, men med betydligt större kvalitetsbortfall än om det omvandlas till WebP, något som kan vara estetiskt mer eller mindre acceptabelt när en ljus eller mörk linjär-gradient läggs på bakgrundsbilden (se förslag i styles.css).
Det etiska förhållningssättet är att få alla att ta del av innehållet på sidan, och även de som använder äldre webbläsare. Modern CSS tillåter även webbläsare som inte kan ladda .webp att läsa .jpg i stället, och optimerad bakgrundsbild i .jpg finns med i image-mappen.

Jag använde även attributet loading="lazy" så att bilden bara laddas när den behövs.

loading="lazy" i html innebär att bilden laddas först efter det att html har laddats ned, vilket är bra för sidans prestanda, det vill säga nedladdningstid **(men sparar inte på energi?)**

Kodoptimering

En annan åtgärd för att minska energiförbrukningen är att minifera mängden data som skickas över nätverket. Detta kan göras genom minifiering av HTML, CSS och JavaScript. Vid minifiering tas onödiga mellanslag, radbrytningar och kommentarer bort från koden, vilket gör filerna mindre. Mindre filer innebär att mindre data behöver överföras mellan server och användare, vilket minskar både laddningstid och energiförbrukning.

(*I den här uppgiften minimiserade jag dock inte koden så att den skulle synas bättre för läraren.)*

Ett annat förslag för att minska miljöpåverkan är att sidan läggs på en server som drivs av göra energi (grön hosting).

Överensstämmelse med annan regelverk/lagstifning

Val av bilder

En personuppgift är allt som kan identifiera en specifik person inklusive kontext enligt General Data Protection Regulation (GDPR). Som exempel valde jag en bild utan personer, eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem. Bilden bör helst passa till den information man vill förmedla, och det kan också vara av symbolisk art. Ur tillgänglighetssynpunkt behöver alla bilder ha en alt-tag.

Bilden på sidan och bakgrundbilden faller under lagen (1960:729) om rätt till litterära och konstnärliga verk (URL). Kan bilderna klassas som konstverk och uppfyller så kallad verkshöjd blir de skyddade som verk. Fotografiet på skrivmaskinen, i den mån fotot är tagen med en kamera kan eventuellt bli skyddad som så kallad närstående rättighet enligt fotorätten i 49 §URL, om fotografiet inte anses vara ett verk. I bägge fall har skaparen en ideell och ekonomisk rätt och ensamrätt att förfoga över sina bilder, och vari ingår att mångfaldiga/kopiera bilder, bearbeta dessa, visa dessa och sprida dessa offentligt i olika medier. Skyddstiden för upphovsrätten gäller 70 år efter upphovsmannens död, och för närstående rättigheten i 50 år efter det att bilden togs.

När en webbläsare laddar ned en bild skapas en kopia av bilden på användarens enhet, och bilden görs samtidigt tillgänglig för webbplatsens besökare.
Licenser är upphovsmännens och fotografers frivilliga avtal om att i mer eller mindre utsträckning erbjuda andra att förfoga över deras bilder utan att betala för det. Det finns olika typer av licenser med såväl friheter som inskränkningar i förfogandet för bilder, filmer, ljud och andra alster. [7]

[7] Se exempel på licenser på https://creativecommons.org/share-your-work/cclicenses/

Man behöver alltid kontrollera om nedladdningsplatsen erbjuder giltiga licenser, och om dessa inte finns be skaparen om lov före användning.

Bilden på sidan och bakgrundsbilden faller under Creative Commons, CC0-licens och de får både kopieras och visas offentligt även i kommersiellt syfte.

Upphovsrätt och närstående rättigheter

Datainsamling via formulär

Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Därför använder formuläret endast ett enkelt meddelandefält och bara de personuppgifter bör samlas som behövs för ändamålet, såsom en första kontakt.

Användning av webbanalys-verktyg, information och samtycke

Enligt ePrivacy-direktivet behöver webbplatsen informera om och inhämta samtycke redan innan ett trafik-analysprogram, t.ex. Google Analytics eller en europeisk motsvarighet såsom Matomo, Plausible Analytics eller Fathom Analytics används. Besökaren behöver aktivt godkänna cookies som inte är att anse nödvändiga, med andra ord ska besökaren kunna neka statistikcookies.
Google Analytics är inte förbjudet enligt GDPR, eftersom programmet nu har automatisk IP-anonymisering, men som med vilket annat webbanalys-verktyg, man måste informera användaren vilka data som samlas in, varför, vem som får tillgång till dem, hur länge de sparas, och hur man kan säga nej.

Information och samtycke

Information till besökaren kan till exempel ske genom en cookie-banner och en integritetspolicy. Där anges vilka uppgifter som samlas in, exempelvis IP-adress, webbläsare, besökta sidor och besökstid. Det räcker dock inte med att informera besökaren. Analysverktyg får inte aktiveras förrän användaren har gett sitt samtycke.

Med andra ord är det först efter det att användaren har godkänt analysverktyg som skriptet från exempelvis Google Tag Manager/gtag.js får köras.

Google Analytics implementeras vanligtvis med JavaScript-biblioteket gtag.js, vilket laddas från Googles servrar. Installationskoden tillhandahålls i Googles officiella utvecklardokumentation.[8] På webbsidan finns dock den enklase JavaScript kod versionen som inte tar hänsyn till att användaren bör samtycka att analysprogrammet körs (i det här projektet exempelvis via ett cookie-samtycke). Skriptet i <head> med hade kunnat aktiverats direkt när sidan laddas utan att användaren har accepterat datainsamlingen, om inte det hade stoppas av Content Security Policy, för då körs det inget analysprogram.

Se i stället förslag på "Alternativ skript" i script.js. Först när användaren klickar på Acceptera så laddas en analystjänst, t.ex. Googles bibliotek och gtag('config') laddar analysprogrammet.

Ökad hänsyn till användaren

Cookie-bannern är utformad som en informationsbanner och inte som en så kallad modal dialog. Det innebär att användaren fortfarande kan navigera vidare på sidan efter att ha sett informationen. En modal dialog skulle istället blockera resten av sidan och kräva att användaren gör ett aktivt val innan fortsatt navigation är möjlig.
WCAG ställer krav på att användaren ska kunna förstå informationen, navigera med tangentbord och ta ett informerat beslut. GDPR (artikel 4.11 och 7) anger samtidigt att samtycke måste vara frivilligt, informerat, specifikt och otvetydigt. Det är till exempel inte tillåtet med förkryssade rutor, dolda val eller lösningar där det är svårare att neka än att acceptera cookies.

Det är däremot juridiskt möjligt att informera användaren och förvänta sig en aktiv handling, till exempel att acceptera eller neka cookies. I EU-praxis gäller dock oftast att användaren ska kunna neka analyscookies och ändå fortsätta använda webbplatsen. Om tillgången till sidan begränsas tills cookies accepteras anses samtycket i många fall inte vara frivilligt.

---

[8] Länk till installationssida från Google Analytics. Set up the Google tag with gtag.js. https://developers.google.com/tag-platform/gtagjs

Det finns även exempel på så kallade *dark patterns*, där designen påverkar användaren att välja ett visst alternativ. Ett exempel är banners som blockerar sidan tills användaren gör ett aktivt val, samtidigt som det är tidskrävande att stänga av många förvalda cookies i inställningarna. Sådana lösningar kan dessutom uppfattas som oetiska och riskerar att försämra användarupplevelsen.

Ur ett användarperspektiv är det därför viktigt att gränssnittet inte upplevs som påträngande eller vilseledande. Även om lagstiftningen tillåter att en användare gör ett aktivt val för att fortsätta använda en webbplats kan det i många fall vara bättre för användarupplevelsen att inte blockera hela sidan med en banner.

Samtidigt bör det tydligt framgå att vissa cookies kan vara tekniskt nödvändiga för att webbplatsen ska fungera. Sådana cookies kräver normalt inte samtycke enligt EU:s cookie-regler.

Utöver detta måste GDPR i övrigt följas. Det innebär bland annat att det måste finnas en rättslig grund för datainsamling, att samtycke till behandling av personuppgifter ska vara frivilligt och aktivt samt att det ska finnas tydliga rutiner för att radera uppgifter om användaren begär det.

När det gäller kontaktformuläret kan insamling av vissa uppgifter vara nödvändig för att kunna svara på användarens meddelande. Informationen om syftet med denna datainsamling kan därför presenteras som ett informativt meddelande, exempelvis: *"Vi kommer att använda dina uppgifter för att svara på ditt meddelande"*, snarare än som ett samtycke som användaren måste godkänna.

Användaren bör också kunna använda formuläret utan att samtidigt ge sitt samtycke till marknadsföring. Checkboxen för marknadsföring: *"Jag godkänner att mina uppgifter används för marknadsföring från Organisation AB", den* bör därför vara avmarkerad från början och inte utformas på ett vilseledande sätt. Attributet required bör inte användas i detta fall, eftersom samtycket ska vara frivilligt. Annars skulle användaren i praktiken tvingas acceptera marknadsföring för att kunna skicka sitt meddelande via formuläret.

Kortfattad reflektion om säkerhet, etik och hållbarhet.

I arbetet med webbplatsen försökte jag balansera säkerhet, tillgänglighet och

hållbarhet.
Ur ett etiskt perspektiv är det viktigt att skydda användarens integritet och inte samla in mer information än nödvändigt. Exempel på insamlade data på den här sidan är: namn, epost och meddelande. Därför använder formuläret endast ett enkelt meddelandefält och lagrar ingen data. Jag valde även en bild utan personer för att undvika att personuppgifter eller identifierbara individer förekommer.
Hänsyn till hållbarhetsaspekten tas genom att optimera bilder och ta bort onödig animation som skulle kunna öka energiförbrukningen.
Jag valde en bild utan personer eftersom fotografier av identifierbara människor kan räknas som personuppgifter. Genom att använda en bild på ett objekt undviks potentiella integritetsproblem.